Design Report

AFS-Like Distributed File System

1 System Model

This report explains the design for a distributed fault tolerant system that finds unique prime numbers and store them in a file server.

- **System Model:** The system is consists of the following (File Servers, Coordinator, Workers) communicating over an asynchronous network. Processes and network links can fail (e.g., crash failures, network partitions).
- **Core Components:**
  1. **File Primary and Replicated Servers:** A replicated, high-availability file server based on the Raft consensus algorithm.
  2. **Application Coordinator:** A central service that manages the task assignments, splits work, and handles failures.
  3. **Application Worker:** A worker process that requests tasks from the Coordinator, performs computation, and interacts with the File Servers.

2 AFS-Like Distributed File System

The file system provides highly-available storage for input datasets and output results as shown in Figure (1).

2.1 Architecture & Core RPCs

The system communications falls under the following RPC commands:

- UploadFile(stream FileChunk): Handles worker file uploads (writes).
- FetchFile(FetchRequest): Handles worker file downloads (reads).
- RequestVote(VoteRequest): Internal Raft RPC for leader election.
- Heartbeat(HeartbeatRequest): Internal Raft RPC for leader authority and liveness.
- ReplicateFile(stream ReplicateFileRequest): Internal RPC for the leader to push file data to followers.

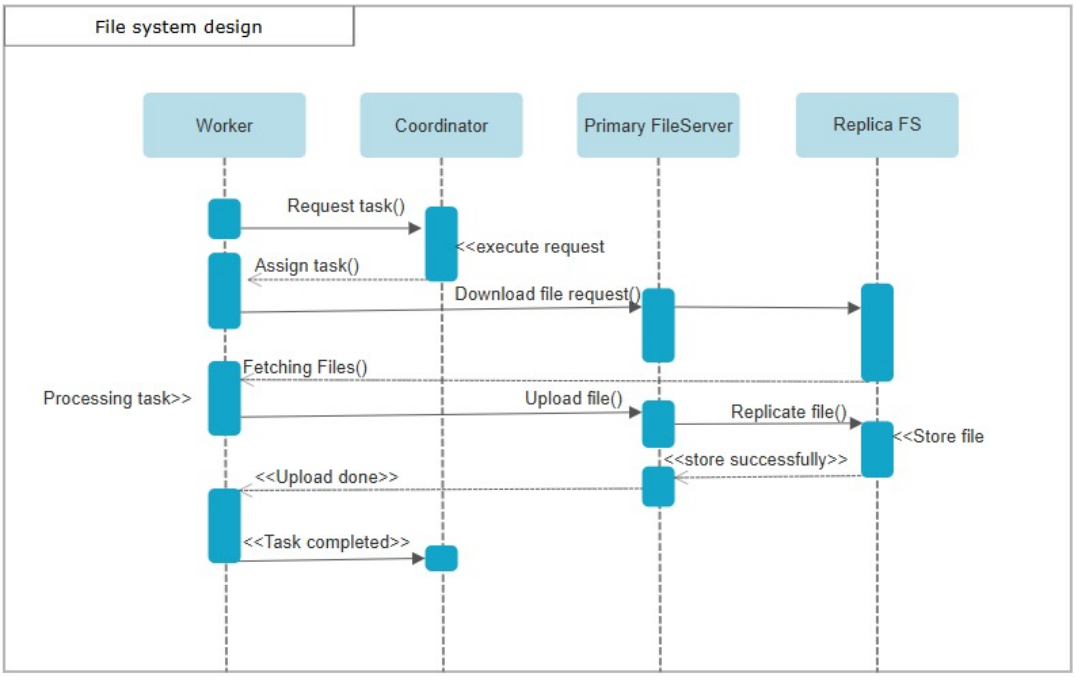


Figure 1: File system design sequence diagram.

2.2 Fault Tolerance

High availability is achieved at multiple levels. For the application, a backup coordinator and task re-assignment logic are used. For the file servers, the **Raft consensus algorithm** is used, not a simple primary-backup.

Application Fault Tolerance

- **Coordinator Failure (Backup Coordinator):** To prevent the Coordinator from being a single point of failure, a backup Coordinator runs in the background. If the primary Coordinator fails (detected via missed heartbeats), the backup takes over. Workers are configured to automatically connect to the backup's address.
- **Worker Failure (Heartbeating):** Workers send periodic heartbeats to the Coordinator. If the Coordinator does not receive a heartbeat from a worker after a set timeout, it marks the worker as 'dead' and the task as 'failed' and assigns it to another worker.
- **Task Re-assignment & Snapshots:** When a task is marked as failed, the Coordinator re-assigns it to a new, available worker. To avoid starting from scratch, workers periodically save their progress to the file system. The new worker loads this snapshot and resumes computation from the last checkpoint.

File Server Fault Tolerance (Raft)

- **Leader Election:** Servers start as (Follower). If no heartbeats are received from a leader within a randomized timeout, a server becomes a (Candidate), increments its term, and requests votes. If it wins a majority, it becomes (Leader).
- **Write Path (Leader-Only):** All write operations (UploadFile) are handled exclusively by the leader. If a worker contacts a follower, the follower rejects the request, and the worker is responsible for retrying with another server until it finds the leader.
- **Replication Path:**
  1. The Leader accepts an (UploadFile) stream from a worker.
  2. It commits the file to its local storage and updates its in-memory file map (version, checksum, etc.).
  3. It then triggers an asynchronous replication to all followers.
  4. The Primary server calls the ReplicateFile RPC on each follower, streaming the new file data to them.
- **Read Path (Read-Optimized):** Any server (Leader or Follower) can service a FetchFile request. This aligns with the read-heavy AFS-like specification, allowing read load to be balanced across all nodes.